

Sijil — Software Architecture & Technical Design

Version: 1.0 (Draft blueprint) **Owner:** Solo founder, technical **Status:** Pre-implementation

1. Project Overview

Project name: Sijil (working name)

Summary: Sijil is an AI-driven trade-compliance platform for UAE importers, exporters, freight forwarders, and customs brokers. It ingests shipment documents (commercial invoice, packing list, bill of lading), predicts correct 12-digit HS codes, cross-checks documents for the inconsistencies that trigger Dubai Customs (Mirsal 2) Yellow/Red-channel review, flags which UAE regulatory permits (MOCCA, MOHAP, TDRA, ESMA/MoIAT, Ministry of Economy strategic-goods control) apply to a given shipment before it's booked, and calculates whether a shipment qualifies for preferential tariff treatment under one of the UAE's CEPA agreements.

Problem being solved: UAE-bound/outbound shipments are delayed or fined because of (a) misclassified HS codes, (b) mismatched data across invoice/packing-list/BOL, (c) missed category-specific permits discovered too late, and (d) missed CEPA duty savings because origin-qualification is done manually or not at all. No existing UAE-localized tool solves all four together; global AI classification tools (Thomson Reuters, Avalara, Digicust) aren't localized to Mirsal 2 or UAE's permit landscape, and UAE freight software (QuickMove, CargoNet) treats documents as storage, not compliance intelligence.

Why the project exists: To reduce customs delays, fines, and duty overpayment for UAE trading companies and to give customs brokers a force-multiplying tool, without requiring a government license to start selling.

Target users:

- Primary: SME/mid-size trading companies (import/export) in UAE, 5–500 employees
- Secondary: Customs clearing agents and freight forwarders acting on behalf of clients
- Tertiary (future): Larger enterprises wanting API integration into their own ERP

Main use cases:

1. A trader uploads an invoice + packing list before booking a shipment and gets an HS

code prediction with confidence score and reasoning.

2. The system flags a data mismatch (e.g., invoice quantity $\neq$ packing list quantity) before the trader files with Mirsal 2.
3. The system tells the trader which permits (if any) apply to their HS code/product category, before they've paid for shipping.
4. The system calculates whether the shipment qualifies for CEPA preferential tariff treatment and what documentation is needed to claim it.
5. A customs broker manages multiple client companies' shipments through one dashboard.

Business/technical objectives:

- Reach a working MVP a solo technical founder can build and maintain
- Get to first 10 paying pilot customers (trading companies / brokers) without needing a government license
- Keep infrastructure cost and operational complexity low enough for one person to run

Primary goals (MVP):

- HS code classification with confidence scoring and human-readable reasoning
- Cross-document consistency checking (invoice / packing list / BOL)
- Permit-applicability triage across the 5 major UAE regulators covered in research (MOCCAE, MOHAP, TDRA, ESMA/MoIAT, Ministry of Economy strategic goods)
- CEPA origin-qualification calculator for the top 3–5 CEPA partner countries by UAE trade volume
- Company/user accounts, document upload, results dashboard, PDF compliance report export

Non-goals (explicitly out of scope for MVP):

- Direct Mirsal 2 filing/submission (read-only advisory tool at MVP; auto-filing is a post-MVP integration requiring broker/customs relationships)
- Payment of duties/VAT on the user's behalf
- Freight booking, carrier rate comparison, container tracking (these are DP World CARGOES / QuickMove territory — not the wedge)
- Full ERP/accounting functionality

- Non-UAE jurisdictions (single-country focus for MVP)
- Physical document notarization/legalization workflows

Scope: Web application (desktop + mobile-responsive), UAE-only, English + Arabic UI, individual company accounts with multi-user support for brokers managing multiple clients.

Assumptions (explicitly labeled per solo-founder, no-confirmed-legal-review context):

- ASSUMPTION: HS classification will start as an AI-assisted *suggestion* tool, not a legally-binding classification service, to avoid regulatory/liability exposure until the product has traction and possibly legal counsel/insurance.
- ASSUMPTION: Permit-triage rules will be maintained as a structured, versioned rules database (not scraped live from ministry sites at request time), updated manually/periodically by the founder or a part-time compliance consultant.
- ASSUMPTION: CEPA calculator covers rules-of-origin logic for a curated subset of agreements at MVP (e.g., India, Indonesia — UAE's largest CEPA trade partners), expanding over time.
- ASSUMPTION: No direct government API access at MVP; all UAE regulatory data is sourced from public documentation and encoded into Sijil's own rules engine.
- ASSUMPTION: Target hosting region is UAE or nearby (Bahrain) for data residency comfort, though not yet a confirmed legal requirement for this specific data type.

Constraints:

- Solo founder — architecture must avoid needing a dedicated DevOps/SRE role
- No enterprise sales cycle at MVP — must be self-serve signup capable
- AI/ML compute costs must stay low relative to a bootstrapped/early-stage budget

Dependencies on external systems/services:

- LLM provider (for document understanding, reasoning, and HS code explanation generation)
- OCR service (for scanned/PDF invoice ingestion)
- Object storage for uploaded documents
- Email/notification provider
- Payment/subscription billing provider

Terminology:

- **HS Code:** Harmonized System code; UAE uses 12-digit codes aligned to GCC Common Customs Tariff
 - **Mirsal 2:** Dubai Customs' electronic declaration system
 - **CEPA:** Comprehensive Economic Partnership Agreement — UAE bilateral trade agreements granting preferential tariffs
 - **Yellow/Red channel:** Mirsal 2's risk-based inspection routing (Green = auto-clear, Yellow = document review, Red = physical inspection)
 - **CoO:** Certificate of Origin
-

2. Functional Requirements

2.1 Document Ingestion & Extraction

- **Purpose:** Accept commercial invoice, packing list, and BOL/AWB as PDF/image/scanned uploads and extract structured data.
- **Actors:** Trader, Broker (on behalf of client)
- **Preconditions:** Authenticated user, active company account
- **Inputs:** PDF/JPG/PNG files (max 20MB each), up to 3 documents per shipment record
- **Processing:** OCR (if scanned) → LLM-based structured field extraction (product description, quantity, unit price, total value, weight, HS code if present, party names/addresses, incoterms) → normalized into a canonical `ShipmentDocument` schema
- **Outputs:** Structured JSON representation per document, confidence score per extracted field
- **Success:** All required fields extracted above a confidence threshold (e.g., 85%)
- **Failure:** Fields below threshold are flagged for manual entry/correction; document marked "needs review"
- **Validation rules:** Numeric fields must parse as numbers; dates must be valid; currency codes must be ISO 4217
- **Edge cases:** Multi-page invoices, handwritten annotations, non-English (Arabic/Chinese) invoices, low-quality scans

- **Permissions:** Only users within the owning company (or their broker) can view/edit

2.2 HS Code Classification

- **Purpose:** Predict the correct 12-digit UAE HS code for each line item
- **Inputs:** Product description, extracted attributes (material, use, technical specs if present), optional product images
- **Processing:** Embedding-based retrieval against a UAE tariff schedule knowledge base + LLM reasoning applying General Rules of Interpretation (GRI 1–6) → ranked candidate codes with confidence and human-readable justification
- **Outputs:** Top-1 recommended code, top-3 alternatives, confidence score, GRI-based reasoning text, applicable duty rate
- **Success condition:** User accepts a suggested code, or manually overrides with their own (both are logged)
- **Failure condition:** Confidence below threshold → explicitly flagged “manual classification recommended,” never silently guessed
- **Edge cases:** Composite/multi-material goods (GRI 3 applies), goods not clearly matching any existing tariff heading, new/novel products
- **Dependencies:** Tariff schedule knowledge base (Section 9), LLM provider

2.3 Cross-Document Consistency Check

- **Purpose:** Catch mismatches between invoice, packing list, and BOL before Mirsal 2 submission
- **Inputs:** Extracted structured data from all uploaded documents for a shipment
- **Processing:** Field-by-field comparison (party names, quantities, weights, values, HS codes if present on multiple docs) with fuzzy matching for name variations (e.g., “LLC” vs “L.L.C.”)
- **Outputs:** List of detected discrepancies, severity level (blocking / warning / informational), suggested resolution
- **Success:** Zero blocking discrepancies before user proceeds
- **Failure:** Blocking discrepancy present → user must resolve or explicitly acknowledge before generating final compliance report
- **Edge cases:** Partial shipments split across multiple BOLs, consolidated cargo with multiple invoices

2.4 Permit Applicability Triage

- **Purpose:** Determine which of the ~10 UAE category-specific regulators require a permit for this shipment
- **Inputs:** HS code(s), product category, product description
- **Processing:** Rules engine lookup against a maintained HS-code-to-regulator mapping table (MOCCAE, MOHAP, TDRA, ESMA/MoIAT, Ministry of Economy strategic goods, Min. of Interior for vehicles, etc.)
- **Outputs:** List of applicable regulators, required permit type, estimated processing time, link/reference to the relevant application portal
- **Success:** Clear "no permits required" or itemized list with next steps
- **Failure/edge case:** HS code ambiguous or spans multiple categories → flag all plausibly-applicable regulators rather than guessing one
- **Dependencies:** Permit rules database (must be kept current — this is an ongoing maintenance responsibility, not a one-time build)

2.5 CEPA Origin Qualification

- **Purpose:** Determine if a shipment qualifies for preferential tariff treatment under an applicable CEPA
- **Inputs:** HS code, declared country of origin, value breakdown (local content vs. imported content, if available), destination/origin country pair
- **Processing:** Rules-of-origin logic per agreement (e.g., regional value content threshold, change-in-tariff-classification test) applied against the specific CEPA's rules for that HS heading
- **Outputs:** Qualifies / does not qualify / insufficient data, required supporting documents (e.g., preferential CoO), estimated duty savings
- **Success:** Clear qualification determination with reasoning
- **Failure:** Insufficient origin/value data → request specific missing inputs from user rather than assuming non-qualification
- **Edge cases:** Goods with multi-country supply chains, agreements with product-specific rules that override the general rule

2.6 Compliance Report Generation

- **Purpose:** Produce a shareable PDF/report bundling classification, consistency check results, permit requirements, and CEPA determination for a shipment
- **Inputs:** All results from 2.1–2.5 for a given shipment record
- **Outputs:** PDF report, downloadable and shareable via link
- **Permissions:** Owning company/broker only

2.7 Company & User Management

- **Purpose:** Multi-tenant account structure supporting both direct trading companies and brokers managing multiple clients
- **Actors:** Company admin, company user, broker admin, broker's managed clients
- **Processing:** Standard RBAC — company admin manages users within their company; broker admin manages multiple company workspaces
- **Edge case:** A broker's client wants to later self-manage their own account — requires an ownership-transfer flow

2.8 Subscription & Billing

- **Purpose:** Self-serve subscription management
- **Processing:** Tiered pricing by shipment volume/company size, handled via third-party billing provider (Stripe or regional equivalent)
- **Edge case:** UAE-specific payment methods and VAT-inclusive invoicing required

3. Non-Functional Requirements

Category	Target	Notes
Performance (classification response)	< 15 seconds per document, end-to-end	ASSUMPTION — no confirmed SLA yet; LLM latency dominates
Availability	99% at MVP (not 99.9%)	ASSUMPTION — solo founder cannot realistically staff 24/7 on-call; be honest with early customers about this
Scalability	Support up to ~500 companies, ~5,000	Revisit if traction exceeds this

	shipments/month at MVP scale	
Security	Encryption in transit (TLS 1.2+) and at rest for all documents	Documents contain commercially sensitive data — non-negotiable
Data residency	Prefer UAE/GCC-region hosting	ASSUMPTION — not yet confirmed as a legal requirement, but a reasonable trust signal for UAE government-adjacent product
Observability	Basic structured logging + uptime monitoring	Full tracing/APM is over-engineering at this stage
Maintainability	Single deployable monolith, not microservices	Solo founder constraint — see Section 42
Accessibility	Basic WCAG AA on core flows	Not a compliance requirement for MVP but good practice

Do not silently invent SLAs beyond what's stated — all numeric targets above are flagged assumptions to be revisited once real usage data exists.

4. System Architecture

Given a solo founder and a full-scope MVP, the architecture is a **modular monolith** — one deployable backend service with clearly separated internal modules, not a microservices mesh. Microservices would add operational burden (multiple deployments, service discovery, distributed tracing) with no benefit at this scale; the modular boundaries inside the monolith preserve the *option* to split services later if a specific module (e.g., classification) needs independent scaling.

```

flowchart TD
    subgraph Client
        WebApp["Web App (Next.js)"]
    end

    subgraph Backend["Sijil Backend (FastAPI monolith)"]
        API["API Layer"]
        Auth["Auth Module"]
        DocIngest["Document Ingestion Module"]
        Classify["HS Classification Module"]
    end

```



```

    Consistency["Consistency Check Module"]
    PermitTriage["Permit Triage Module"]
    CEPA["CEPA Origin Module"]
    Reports["Report Generation Module"]
    Billing["Billing Module"]
end

subgraph AsyncLayer["Async Processing"]
    Queue["Task Queue (Redis + Celery)"]
    Worker["Background Workers"]
end

subgraph Data
    DB["PostgreSQL"]
    ObjStore["Object Storage (S3-compatible)"]
    VectorDB["Vector Store (pgvector)"]
end

subgraph External
    LLM["LLM Provider API"]
    OCR["OCR Service"]
    Stripe["Billing Provider"]
    Email["Email/Notification Provider"]
end

WebApp --> API
API --> Auth
API --> DocIngest
API --> Reports
API --> Billing
DocIngest --> Queue
Queue --> Worker
Worker --> OCR
Worker --> Classify
Worker --> Consistency
Worker --> PermitTriage
Worker --> CEPA
Classify --> LLM
Classify --> VectorDB
Worker --> DB
DocIngest --> ObjStore
Billing --> Stripe
API --> Email
Auth --> DB
Reports --> DB

```

Component responsibilities:

Component	Responsibility	Failure behavior	Scaling
API Layer	Request validation, auth, routing to modules	Returns structured error responses; never crashes on bad input	Stateless — scale horizontally behind a load balancer if needed
Document Ingestion	Accepts uploads, stores raw files, queues processing	Retries transient OCR/upload failures; surfaces permanent failures to user	Queue-based, naturally scales via worker count
Classification Module	HS code prediction	Falls back to “manual review required” rather than guessing under low confidence	Bottlenecked by LLM API rate limits — cache/batch where possible
Consistency Module	Field comparison logic	Pure computation, no external dependency — very low failure risk	Trivial to scale, CPU-bound only
Permit Triage Module	Rules engine lookup	Data-driven; failure mode is “rules table out of date,” not a runtime crash	Trivial to scale
CEPA Module	Origin qualification logic	Same as above — data-driven rules engine	Trivial to scale
Report Generation	PDF assembly	Retries on render failure	CPU-bound, run as background job
Billing Module	Subscription state, usage metering	Delegates actual payment processing to Stripe; never stores raw card data	N/A — third-party dependency

5. Architecture Layers

Presentation Layer → Next.js frontend

↓

API / Controller Layer → FastAPI route handlers (thin — validation + delegatio

↓
Service / Business Logic → Classification, Consistency, PermitTriage, CEPA, Repo
↓
Repository / Data Access → SQLAlchemy repositories, one per aggregate (Company,
↓
Database / Storage Layer → PostgreSQL + pgvector, S3-compatible object storage

Rules:

- Controllers never contain business logic — they validate input, call a service, return the result.
 - Services never touch the ORM directly — they call repositories.
 - Repositories never contain business logic — they translate between domain objects and database rows.
 - Dependency direction is strictly downward. A repository must never import a service. A controller must never bypass a service to call a repository directly.
 - Background workers call services directly (not through the API layer) since they run in-process within the same codebase.
-

6. Detailed Component Design

6.1 Classification Service

- **Responsibility:** Given normalized product data, return ranked HS code candidates with reasoning.
- **Inputs:** `ProductLineItem` (description, attributes, optional image reference)
- **Outputs:** `ClassificationResult` (candidates list, confidence, reasoning, applied GRI rules)
- **Interfaces:** `classify(line_item: ProductLineItem) -> ClassificationResult`
- **Dependencies:** `TariffKnowledgeBaseRepository` (vector search), `LLMClient`
- **Internal logic:** (1) embed product description, (2) vector-search top-20 candidate tariff headings from the knowledge base, (3) pass candidates + product data to LLM with a GRI-reasoning prompt, (4) parse structured LLM output into ranked results, (5) apply a confidence floor — below threshold, mark `requires_manual_review=True`
- **State:** Stateless per call; results persisted by the caller

- **Error handling:** LLM timeout → retry once with backoff, then fail gracefully with `requires_manual_review=True` and a clear error message (never silently return a guessed code)
- **Edge cases:** Composite goods, no strong vector match (novel product) — both surface as low-confidence rather than a false-confident guess
- **Security:** Product descriptions may contain commercially sensitive info — ensure LLM provider contract includes no-training-on-data terms
- **Testing:** Golden-set regression tests against a curated list of ~200 known product→HS-code pairs; track accuracy over time as the prompt/knowledge base evolves
- **Extensibility:** Designed to swap LLM providers via the `LLMClient` interface without touching calling code

6.2 Consistency Check Service

- **Responsibility:** Compare structured data across a shipment's documents
- **Inputs:** List of `ExtractedDocument` objects for one shipment
- **Outputs:** List of `Discrepancy` (field, documents involved, severity, suggested resolution)
- **Internal logic:** Field-by-field comparison with type-aware matching (exact match for HS codes/values; fuzzy string match with a similarity threshold for names/addresses)
- **Error handling:** Missing field in one document is a `warning`-severity discrepancy, not a blocking error — some fields legitimately don't appear on every document type
- **Testing:** Unit tests per comparison rule; a suite of known-good and known-bad document sets

6.3 Permit Triage Service

- **Responsibility:** Map HS code(s) to applicable UAE regulators
- **Inputs:** HS code(s), optional product category override
- **Outputs:** List of `PermitRequirement` (regulator, permit type, processing time estimate, reference link)
- **Internal logic:** Lookup against `PermitRuleRepository`, a data table keyed by HS code range/prefix, maintained independently of application code deploys (see Section 23)
- **Data ownership:** This module's accuracy is only as good as the maintained rules table

— this is flagged explicitly as an ongoing operational responsibility, not a one-time engineering task

- **Testing:** Table-driven tests against known HS-code-to-regulator mappings from research (food → MOCCAE, telecom → TDRA, etc.)

6.4 CEPA Origin Service

- **Responsibility:** Determine preferential tariff eligibility
 - **Inputs:** HS code, origin country, destination country, value-content breakdown (optional/partial allowed)
 - **Outputs:** `OriginDetermination` (qualifies/does not/insufficient-data, reasoning, required documents, estimated savings)
 - **Internal logic:** Per-agreement rule lookup (regional value content % vs. change-in-tariff-classification test, depending on the specific CEPA and HS heading)
 - **Edge case handling:** If value-content data is missing, explicitly request it rather than defaulting to “does not qualify” — a false negative costs the user real money
 - **Testing:** Rule-specific unit tests per covered CEPA agreement
-

7. Codebase / Repository Architecture

```
sijil/
├── backend/
│   ├── app/
│   │   ├── api/                                # FastAPI route handlers (thin controllers)
│   │   │   ├── v1/
│   │   │   │   ├── shipments.py
│   │   │   │   ├── classification.py
│   │   │   │   ├── companies.py
│   │   │   │   ├── auth.py
│   │   │   │   └── billing.py
│   │   ├── services/                          # Business logic – one file per domain service
│   │   │   ├── classification_service.py
│   │   │   ├── consistency_service.py
│   │   │   ├── permit_triage_service.py
│   │   │   ├── cepa_service.py
│   │   │   ├── report_service.py
│   │   │   └── billing_service.py
│   │   ├── repositories/                      # Data access – one file per aggregate
│   │   │   └── shipment_repository.py
```

```

| | | | └─ document_repository.py
| | | | └─ company_repository.py
| | | | └─ tariff_kb_repository.py
| | | | └─ permit_rules_repository.py
| | | └─ models/ # SQLAlchemy ORM models
| | | └─ schemas/ # Pydantic request/response schemas
| | | └─ workers/ # Celery task definitions
| | | | └─ document_processing_tasks.py
| | | | └─ report_generation_tasks.py
| | | └─ integrations/ # External service clients
| | | | └─ llm_client.py
| | | | └─ ocr_client.py
| | | | └─ billing_client.py
| | | | └─ email_client.py
| | | └─ middleware/ # Auth, request logging, error handling
| | | └─ config/ # Settings, environment config
| | | └─ utils/
| └─ tests/
| | └─ unit/
| | └─ integration/
| | └─ fixtures/
| | └─ golden_sets/ # HS classification regression test data
| └─ alembic/ # DB migrations
| └─ pyproject.toml
└─ frontend/
| └─ app/ # Next.js app router pages
| └─ components/
| └─ lib/
| └─ package.json
└─ data/
| └─ permit_rules/ # Versioned permit-rule data files (see Secti
└─ docs/
| └─ PRD.md
| └─ SOFTWARE_ARCHITECTURE_AND_TECHNICAL_DESIGN.md
| └─ API_SPEC.md

```

Directory rules:

- `api/` may import from `services/` and `schemas/` only — never directly from `repositories/` or `models/`.
- `services/` may import from `repositories/` and `integrations/` — never directly from `models/` (`repositories` translate models into domain objects `services` work with).
- `repositories/` may import from `models/` — never from `services/` (would create a circular dependency).

- `workers/` may import from `services/` directly (they act as an alternate entry point, parallel to `api/`).
-

8. Module / Class / Interface Design

Key abstractions — used only where they provide real value (LLM/OCR swapping, testability), not introduced speculatively:

```
# integrations/llm_client.py
class LLMClient(Protocol):
    def classify_product(self, description: str, candidates: list[TariffHeading])
    def extract_document_fields(self, raw_text: str, doc_type: DocumentType) -> E
```

This interface exists specifically so the LLM provider can be swapped (e.g., if pricing/quality changes) without touching `classification_service.py` or `document_processing_tasks.py`. Everything else (repositories, most services) is implemented concretely — introducing interfaces for single-implementation internal classes would be premature abstraction per Section 42's guidance.

```
classDiagram
    class ClassificationService {
        +classify(line_item) ClassificationResult
    }
    class LLMClient {
        <<interface>>
        +classify_product()
        +extract_document_fields()
    }
    class TariffKBRepository {
        +vector_search(query) list~TariffHeading~
    }
    ClassificationService --> LLMClient
    ClassificationService --> TariffKBRepository
```

9. Data Architecture

Database: PostgreSQL with the `pgvector` extension for tariff-heading embedding search — avoids running a separate vector database (e.g., Pinecone) at this scale, reducing operational surface area for a solo founder. Revisit only if vector search volume/latency

demands a dedicated engine.

erDiagram

```
COMPANY ||--o{ USER : employs
COMPANY ||--o{ SHIPMENT : owns
BROKER_COMPANY ||--o{ COMPANY : manages
SHIPMENT ||--o{ DOCUMENT : contains
SHIPMENT ||--o{ LINE_ITEM : contains
LINE_ITEM ||--o{ CLASSIFICATION_RESULT : has
SHIPMENT ||--o{ DISCREPANCY : has
SHIPMENT ||--o{ PERMIT_REQUIREMENT : has
LINE_ITEM ||--o{ ORIGIN_DETERMINATION : has
SHIPMENT ||--o{ REPORT : generates
COMPANY ||--o{ SUBSCRIPTION : has
```

```
COMPANY {
    uuid id PK
    string legal_name
    string trade_license_number
    uuid broker_company_id FK
    timestamp created_at
}
```

```
USER {
    uuid id PK
    uuid company_id FK
    string email
    string role
    timestamp created_at
}
```

```
SHIPMENT {
    uuid id PK
    uuid company_id FK
    string status
    string direction
    timestamp created_at
}
```

```
DOCUMENT {
    uuid id PK
    uuid shipment_id FK
    string doc_type
    string storage_path
    jsonb extracted_fields
    float extraction_confidence
}
```

```
LINE_ITEM {
    uuid id PK
```



```

        uuid shipment_id FK
        string description
        decimal quantity
        decimal unit_value
        string currency
    }
    CLASSIFICATION_RESULT {
        uuid id PK
        uuid line_item_id FK
        string hs_code
        float confidence
        text reasoning
        boolean requires_manual_review
        string user_override_code
    }
    DISCREPANCY {
        uuid id PK
        uuid shipment_id FK
        string field
        string severity
        text description
    }
    PERMIT_REQUIREMENT {
        uuid id PK
        uuid shipment_id FK
        string regulator
        string permit_type
        text reference_link
    }
    ORIGIN_DETERMINATION {
        uuid id PK
        uuid line_item_id FK
        string agreement
        boolean qualifies
        text reasoning
    }
    REPORT {
        uuid id PK
        uuid shipment_id FK
        string storage_path
        timestamp generated_at
    }
    SUBSCRIPTION {
        uuid id PK
        uuid company_id FK
        string plan
        string status
    }

```

```
    string billing_provider_ref
}
```

Key constraints:

- `Shipment.company_id` — foreign key, indexed, non-null (every shipment must belong to a company)
- `ClassificationResult.confidence` — check constraint `>= 0 AND <= 1`
- `Document.extraction_confidence` — same
- Unique constraint on `(Company.trade_license_number)` — prevent duplicate company registration

Query patterns to optimize for:

- “All shipments for a company, most recent first” — index on `(company_id, created_at DESC)`
- “All line items needing manual classification review” — partial index on `requires_manual_review = true`
- Vector similarity search on tariff knowledge base embeddings — `pgvector` IVFFlat or HNSW index

Data retention: Shipment documents retained for at least 5 years (aligns with UAE VAT record-keeping requirement mentioned in research for export zero-rating evidence) — ASSUMPTION, not yet confirmed against final legal requirements for this specific product.

Caching: Tariff knowledge base is read-heavy/write-rare — cache vector search results for identical/near-identical queries at the application layer (simple TTL cache) rather than introducing Redis-based caching at MVP scale beyond what Celery already requires.

10. API Architecture

Base URL: `/api/v1` . REST, JSON, JWT bearer auth on all endpoints except `/auth/*` .

Method	Route	Purpose	Auth
POST	<code>/auth/register</code>	Company + first user signup	None
POST	<code>/auth/login</code>	Returns JWT	None

POST	/shipments	Create a shipment record	Required
POST	/shipments/{id}/documents	Upload a document (multipart)	Required
GET	/shipments/{id}	Get shipment status + all results	Required
GET	/shipments/{id}/classification	Get classification results	Required
POST	/shipments/{id}/line-items/{id}/override	User overrides an HS code suggestion	Required
GET	/shipments/{id}/discrepancies	Get consistency check results	Required
GET	/shipments/{id}/permits	Get permit triage results	Required
GET	/shipments/{id}/origin	Get CEPA determination	Required
POST	/shipments/{id}/report	Trigger report generation (async)	Required
GET	/reports/{id}	Download generated report	Required
GET	/companies/{id}/users	List company users (admin only)	Required, role-checked
POST	/billing/checkout	Initiate subscription checkout	Required

Example — document upload response:

```
{
  "document_id": "doc_8f3a...",
  "status": "processing",
  "estimated_completion_seconds": 12
}
```

Example — classification result:

```
{
  "line_item_id": "li_2b1c...",
  "top_candidate": {
    "hs_code": "8517.62.00.90",
    "confidence": 0.91,
    "reasoning": "Product described as a network switch; classified under 8517.62",
  },
  "alternatives": [{"hs_code": "8471.80.00.00", "confidence": 0.34}],
  "requires_manual_review": false
}
```

Status codes: 200/201 success, 400 validation error, 401 unauthenticated, 403 unauthorized (wrong company/role), 404 not found, 422 unprocessable (e.g., unreadable document), 429 rate-limited, 500 unexpected.

Async pattern: Document processing and report generation are long-running — endpoints return `202 Accepted` with a status URL; frontend polls or uses a WebSocket/SSE channel for live status updates (SSE recommended over WebSocket for simplicity at this scale — one-directional server-to-client updates are sufficient, no need for full-duplex).

Idempotency: Document upload endpoint accepts an `Idempotency-Key` header to prevent duplicate processing on client retry.

Rate limiting: Per-company rate limits on document processing to control LLM cost exposure — ASSUMPTION on exact numbers, to be tuned against real LLM API costs.

11. Core Workflows

11.1 Shipment Document Processing (primary workflow)

```
sequenceDiagram
    participant U as User
    participant API as API Layer
    participant Q as Task Queue
    participant W as Worker
    participant OCR as OCR Service
    participant LLM as LLM Provider
    participant DB as Database

    U->>API: POST /shipments/{id}/documents (upload invoice)
    API->>DB: Store document metadata (status=queued)
    API->>Q: Enqueue processing task
```

```

API-->>U: 202 Accepted + document_id
Q->>W: Dequeue task
W->>OCR: Extract raw text (if scanned)
OCR-->>W: Raw text
W->>LLM: Extract structured fields
LLM-->>W: Structured fields + confidence
W->>DB: Store extracted fields
W->>DB: Update status=extracted
Note over W: If all shipment docs extracted, trigger next stage
W->>Q: Enqueue classification + consistency-check tasks
Q->>W: Dequeue classification task
W->>LLM: Classify each line item
LLM-->>W: HS code candidates + reasoning
W->>DB: Store classification results
W->>DB: Update status=classified
U->>API: GET /shipments/{id} (poll or SSE)
API->>DB: Fetch current status + results
API-->>U: Results

```

Failure path: If OCR or LLM call fails after retry, the specific document/line-item is marked `status=needs_manual_review` with an error reason — the rest of the shipment’s processing continues rather than failing the entire batch.

11.2 Permit Triage + CEPA (triggered after classification completes)

Runs as a fast, synchronous-feeling step (data-driven, no external API calls) immediately following classification within the same worker task chain — no need for separate queue hops given its low latency.

12. Asynchronous Processing and Background Jobs

Job	Trigger	Sync or Async	Why
Document OCR + extraction	Document upload	Async	LLM/OCR latency (5-15s) — unacceptable to block the HTTP request
HS classification	Extraction complete	Async	Same reason; chained via Celery task
Consistency check	All shipment docs extracted	Async → but trivial compute, fast once triggered	Depends on multiple documents being ready

Permit triage	Classification complete	Async (chained)	Depends on classification output
CEPA determination	Classification complete	Async (chained)	Same
Report generation	User-triggered	Async	PDF rendering + assembling all results takes a few seconds
Subscription webhook processing	Stripe webhook	Async	Never block on external payment provider callbacks

Retry policy: Transient failures (network timeout) retry 3x with exponential backoff. Permanent failures (malformed file, unsupported document type) do not retry — surfaced immediately to the user.

Idempotency: Each task keyed by `document_id / shipment_id + task type`; re-running a completed task is a no-op unless explicitly forced (e.g., user requests re-classification).

Dead-letter handling: Tasks failing all retries move to a dead-letter queue with alerting (even a simple email-to-founder alert is sufficient at this scale — no need for a full incident-management system).

13. External Services and Integrations

Service	Purpose	Integration	Failure behavior	Fallback
LLM Provider	Document extraction, HS classification reasoning	REST API	Retry 3x, then mark item for manual review	None at MVP — this is a hard dependency; mitigated by not blocking the rest of the pipeline on one failed item
OCR Service	Scanned document text extraction	REST API	Retry, then request user to upload a clearer scan	None
Object Storage (S3-	Raw document	SDK	Retry; if persistent, block upload with clear	None

compatible)	storage		error	
Billing Provider (Stripe or regional equivalent)	Subscription management	SDK + webhooks	Webhook retries handled by provider; verify webhook signatures	Manual invoicing as a stopgap for early pilot customers
Email Provider	Notifications	REST API	Retry; non-blocking — a failed notification email should never block a user-facing action	None critical

Mocking strategy for development: All external clients implemented behind interfaces (`LLMClient`, `OCRClient`, etc.) with mock implementations for local dev/testing — avoids burning real API credits or requiring live credentials during development.

14. Authentication and Authorization

Authentication: JWT-based, issued on login, short-lived access token (15 min) + refresh token (7 days) stored as an `httpOnly` cookie.

Authorization model — roles:

- `company_admin` — full access within their company, manages users
- `company_user` — can create/view shipments within their company
- `broker_admin` — full access within their own broker company, plus access to managed client companies
- `broker_user` — access to specifically-assigned managed client companies only

Resource-level authorization: Every shipment/document/report query is scoped by `company_id` derived from the authenticated user's token — never trust a client-supplied `company_id`. Broker access to a managed company is checked via an explicit `broker_company_id` relationship, not inferred.

Tenant isolation: Enforced at the repository layer — every query method requires a

`company_id` (or resolved list of accessible `company_id`s for brokers) as a mandatory parameter; there is no "query all shipments" method without a tenant scope, by design, to prevent accidental cross-tenant data leaks.

Service-to-service: Workers run in-process with the same codebase/database credentials — no separate service auth needed at this architecture scale.

Secrets management: Environment variables at MVP, loaded via a validated settings module; move to a managed secrets service (e.g., cloud provider's secrets manager) once team size or compliance requirements grow.

15. Security Architecture

- **Input validation:** All API inputs validated via Pydantic schemas; file uploads validated by MIME type and magic-byte sniffing (not just file extension) before processing
- **Encryption in transit:** TLS enforced everywhere, HSTS enabled
- **Encryption at rest:** Database and object storage encryption enabled at the infrastructure level (managed by cloud provider)
- **Secure file handling:** Uploaded files scanned for malware before processing (a basic AV scan step in the ingestion worker); files stored with non-guessable keys, never served directly from a public bucket
- **Injection risks:** SQLAlchemy ORM with parameterized queries throughout — no raw SQL string interpolation
- **Prompt injection risk:** Since extracted document text is passed into LLM prompts, a malicious/malformed document could attempt prompt injection to manipulate classification output. Mitigation: structure prompts so document content is clearly delimited and treated as data, never as instructions; validate that LLM output conforms to an expected schema (reject/flag anything that doesn't parse as valid structured output)
- **Sensitive data handling:** Company trade documents are commercially sensitive — access logged, never included in application logs in raw form, LLM provider contract should specify no training on submitted data
- **Rate limiting/abuse prevention:** Per-company and per-IP rate limits on upload and auth endpoints
- **Dependency security:** Automated dependency vulnerability scanning in CI (see Section 25)

Lightweight threat model — top risks:

- 1. Cross-tenant data leak via a missing `company_id` filter → mitigated by repository-layer enforcement (Section 14)
- 2. Prompt injection via malicious document content → mitigated by prompt structuring + output validation
- 3. Stolen JWT → mitigated by short expiry + refresh rotation
- 4. Uploaded malware → mitigated by AV scanning before processing

16. Error Handling

Response format (consistent across all endpoints):

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human-readable description",
    "details": {}
  }
}
```

Category	HTTP Status	Retryable?
Validation error	400	No — fix input
Auth error	401	No — re-authenticate
Authorization error	403	No
Not found	404	No
Unprocessable (bad document)	422	No — user must fix input
Dependency failure (LLM/OCR down)	502/503	Yes, with backoff
Rate limited	429	Yes, after <code>Retry-After</code>
		Logged for investigation; user sees generic

Unexpected	500	message, never a stack trace
------------	-----	------------------------------

Where errors are caught: Integration clients (`LLMClient` , `OCRClient`) catch and translate provider-specific exceptions into domain exceptions. Services catch domain exceptions and decide retry vs. fail. Controllers catch any remaining exception and translate to the standard error response format — nothing unhandled should ever reach the client as a raw stack trace.

17. Logging, Monitoring, and Observability

- **Structured logging:** JSON logs with `request_id` , `company_id` (where applicable), `user_id` , `timestamp` , `level`
 - **What to log:** Request/response metadata, processing stage transitions, errors with context, classification confidence distributions (for monitoring model quality drift over time)
 - **What to NEVER log:** Raw document content, full JWT tokens, payment card details, raw LLM prompts containing sensitive commercial data (log a reference/hash instead)
 - **Health checks:** `/health` (liveness) and `/health/ready` (readiness — checks DB and queue connectivity)
 - **Metrics to track from day one:** Classification confidence distribution, manual-review rate, document processing latency, LLM API cost per shipment
 - **Alerting:** At solo-founder scale, a simple alert (email/SMS/Slack webhook) on: dead-letter queue growth, error rate spike, health check failure. Full APM/tracing tooling is over-engineering here — revisit if/when a team is in place.
-

18. Performance and Scalability

Likely bottlenecks:

- LLM API latency/cost — dominant cost driver and latency source; mitigate with caching identical/near-identical classification queries, and batching where the LLM provider supports it
- Vector search on the tariff knowledge base — indexed properly (HNSW/IVFFlat via pgvector), should stay fast at this data volume (tariff schedule is a few thousand

headings, not millions)

- Document upload/OCR for large multi-page PDFs — mitigate with async processing (already designed for) and reasonable file size limits

What to optimize now: Async processing for anything touching an external API (already in the design), database indexes on the query patterns identified in Section 9.

What to defer: Horizontal auto-scaling, read replicas, CDN, advanced caching layers — none of these are justified at ~500 companies / 5,000 shipments a month. A single reasonably-sized application server + managed Postgres will comfortably handle this.

19. Technology Decisions

Decision	Choice	Why	Alternatives rejected
Backend language/framework	Python + FastAPI	Strong LLM/ML ecosystem fit (matches founder's ML background), async support for I/O-bound workloads, fast to build with, excellent typing via Pydantic	Node/TS (weaker ML-ecosystem fit for this specific product); Django (heavier, less natural for an API-first + async-heavy service)
Database	PostgreSQL + pgvector	One database for both relational and vector-search needs, avoids running a separate vector DB, mature, well-understood operationally for a solo founder	Dedicated vector DB (Pinecone/Weaviate) — unnecessary operational complexity at this scale; MongoDB — relational integrity (foreign keys, constraints) matters more here than schema flexibility
Task queue	Celery + Redis	Mature, well-documented Python-native async task processing	Direct use of a managed queue (SQS) — adds cloud-provider lock-in and complexity without a corresponding benefit yet
		Good developer	Plain React SPA — Next.js

Frontend	Next.js (React)	velocity, server-side rendering for a marketing/signup flow, large ecosystem	gives SSR/SEO benefits for the public marketing pages at minimal extra cost
LLM provider	Anthropic Claude API (or similar)	Strong structured-output and reasoning capability needed for GRI-based classification reasoning	Self-hosted open-weight model — infeasible cost/ops burden for a solo founder at this stage
Object storage	S3-compatible (AWS S3 or a UAE/GCC-region equivalent)	Standard, cheap, reliable	Self-hosted storage — unnecessary operational burden
Deployment	Single-region managed cloud (containerized)	Matches solo-founder operational capacity	Kubernetes — significant overkill at this scale (Section 42)
Billing	Stripe (if UAE-supported) or a regional payment gateway	Reduces PCI compliance burden by never touching raw card data	Building custom billing — unnecessary risk and effort

20. Architecture Decision Records

ADR-001: Modular Monolith over Microservices

- **Context:** Solo founder building a full-scope MVP (classification + consistency + permits + CEPA)
- **Options considered:** Microservices per module; modular monolith; serverless functions per capability
- **Chosen approach:** Modular monolith with clear internal module boundaries
- **Reasons:** One person cannot operate multiple deployed services, service discovery, and distributed debugging while also building product features

- **Trade-offs:** Slightly harder to independently scale one module later (e.g., if classification needs 10x more compute than everything else)
- **Consequences:** Module boundaries (Section 7) are designed so a future split into a separate service is possible without a full rewrite, if ever justified by actual load

ADR-002: Advisory Tool, Not Direct Mirsal 2 Filer, at MVP

- **Context:** Direct government system integration requires certification/approval processes with uncertain timelines (mirrors the pattern noted in the OT-security business research — government-facing integrations are slow to establish)
- **Options considered:** Build direct Mirsal 2 filing integration from day one; build an advisory/preparation tool first
- **Chosen approach:** Advisory tool — user still files in Mirsal 2 themselves, using Sijil's output as preparation
- **Reasons:** No waiting on government approval to start selling; faster time to first paying customer
- **Trade-offs:** Less "sticky" than a direct filing integration would eventually be
- **Consequences:** Direct filing integration becomes a natural, valuable post-MVP milestone once the product has traction and credibility

ADR-003: Rules-Engine (not live-scraped) Permit Triage

- **Context:** Permit requirements come from ~10 different UAE government sources with no unified API
- **Options considered:** Scrape government sites live per request; maintain a versioned internal rules database
- **Chosen approach:** Versioned internal rules database, manually maintained
- **Reasons:** Live scraping is fragile, slow, and government sites can change/block scraping; predictable data quality matters more than real-time freshness here
- **Trade-offs:** Requires ongoing manual maintenance discipline — explicitly flagged as an operational responsibility, not solved by engineering alone
- **Consequences:** Section 23 defines how this rules data is versioned and updated

21. Testing Architecture

```

tests/
├─ unit/                # Pure logic – consistency rules, permit rules, CEPA r
├─ integration/         # Service + repository + real test DB
├─ e2e/                 # Full API flows against a test environment
├─ golden_sets/         # HS classification accuracy regression (curated pro
├─ fixtures/           # Sample documents (invoice/packing-list/BOL test fi
└─ helpers/

```

- **Unit tests:** Every rules-engine function (permit triage, CEPA logic, consistency comparison) — these are pure functions, ideal for exhaustive unit testing
 - **Integration tests:** Service-to-repository-to-database flows using a real (containerized) test Postgres instance
 - **E2E tests:** Critical user journeys (signup → upload → classification → report) run against a staging environment
 - **Golden-set regression tests:** A maintained set of ~200+ known-correct product-description-to-HS-code pairs, run on every change to the classification prompt/logic to catch accuracy regressions — this is the single most important test suite in the system, since classification quality is the product's core value
 - **Mocking:** All external services (LLM, OCR, billing) mocked in unit/integration tests; only E2E/staging tests hit real (sandboxed) external APIs
-

22. Development Environment

- **Prerequisites:** Python 3.12+, Node 20+, Docker (for local Postgres + Redis)
- **Local services:** `docker-compose up` for Postgres + Redis; backend and frontend run natively for fast reload
- **Environment variables:** `.env.local` (gitignored), validated on startup via a Pydantic settings class — app fails fast with a clear error if a required variable is missing, rather than failing mysteriously later
- **Seed data:** A seed script populating a sample company, user, and a few example shipments with mocked classification results, for local UI development without needing live LLM calls
- **Migrations:** Alembic, run automatically on backend startup in dev, manually gated in production (see Section 25)
- **Debugging:** Standard debugger + structured local logs; no exotic tooling needed at this

23. Configuration Management

- **Config vs. secrets:** Non-sensitive config (feature flags, timeout values) in a checked-in `config/settings.py` with environment overrides; secrets (API keys, DB credentials) exclusively via environment variables, never committed
 - **Permit rules data:** Stored as versioned JSON/YAML files under `data/permit_rules/`, checked into version control with a clear changelog — this makes rule updates auditable and reviewable (important given these rules directly affect legal compliance advice given to users) rather than being an opaque database edit
 - **Environment-specific settings:** `dev / staging / production` configs differ in LLM provider keys (sandbox vs. live), database connection, and log verbosity
 - **Validation:** App startup validates all required config is present and well-formed; refuses to start otherwise
-

24. Deployment Architecture

```
flowchart TD
    subgraph CDN_Edge ["Edge"]
        CDN["CDN / Static Assets (Next.js)"]
    end
    subgraph Compute ["Application Layer"]
        App["Backend App Container (FastAPI)"]
        Worker["Worker Container (Celery)"]
    end
    subgraph Managed ["Managed Services"]
        PG["Managed PostgreSQL"]
        Redis["Managed Redis"]
        S3["Object Storage"]
    end
    Users --> CDN
    Users --> App
    App --> PG
    App --> Redis
    Worker --> Redis
    Worker --> PG
    Worker --> S3
    App --> S3
```

- **Environments:** dev (local), staging (mirrors production, used for E2E tests + manual QA before release), production
 - **Containers:** Single Docker image for the backend, run as two container types (API process, worker process) from the same image with different entrypoints — reduces build/maintenance duplication
 - **Infrastructure:** A single managed container hosting platform (e.g., a PaaS like Render/Railway, or a single cloud VM/ECS setup) — explicitly avoiding Kubernetes (Section 42)
 - **Region:** UAE/GCC-region hosting preferred per the data-residency assumption in Section 1
-

25. CI/CD

Pipeline steps (in order):

1. Lint + format check (ruff/black for Python, eslint/prettier for frontend)
2. Type check (mypy / TypeScript)
3. Unit tests
4. Integration tests (against a spun-up test Postgres/Redis in CI)
5. Golden-set classification regression check (fails the build if accuracy drops below a set threshold vs. the last known-good baseline)
6. Dependency vulnerability scan
7. Build container image
8. Deploy to staging automatically on merge to `main`
9. Manual promotion to production (explicit approval step — no fully automatic production deploys at this stage, given solo-founder risk tolerance)
10. Database migrations run as a separate, gated step before the new app version receives traffic — never auto-run destructive migrations without review

Rollback strategy: Keep the previous container image tagged and ready; rollback is “redeploy the last known-good image” — no need for a more elaborate blue/green setup at this scale, though the deployment platform chosen should support at least basic instant-

rollback.

26. Infrastructure and Production Concerns

- **DNS/TLS:** Standard managed DNS + automatic TLS certificate provisioning (e.g., via the hosting platform or Let’s Encrypt)
- **Backups:** Managed Postgres automated daily backups with point-in-time recovery, provided by the hosting platform — do not build custom backup tooling
- **High availability:** Not justified at MVP scale (Section 3) — single-instance app + managed DB with automated failover (if the managed DB provider offers it) is sufficient
- **Autoscaling:** Not needed yet; revisit if traffic patterns demand it post-traction
- Kubernetes, multi-region, and serverless are explicitly NOT used — none are justified by current requirements (Section 42)

27. Backup and Disaster Recovery

Data	Backup approach	RPO	RTO
PostgreSQL (all structured data)	Automated daily + point-in-time recovery via managed provider	< 24 hours (ideally near-zero via PITR)	A few hours
Object storage (uploaded documents)	Provider-level redundancy (e.g., S3 durability guarantees) + versioning enabled	Near-zero	Minutes
Permit rules data	Version-controlled in git — recoverable from repository history	Zero (it’s source code)	Minutes

Critical data: Shipment documents and classification results (legal/financial consequence for users if lost). **Non-critical:** Cached vector search results, application logs.

28. Repository-Level Dependency Graph

api/ → services/ → repositories/ → models/

```
→ integrations/  
workers/ → services/ (same as above)
```

No circular dependencies by construction — enforced via the directory import rules in Section 7. A CI lint rule (e.g., `import-linter` for Python) should be added to mechanically enforce these boundaries rather than relying on developer discipline alone, since a solo founder moving fast is exactly the situation where these rules get accidentally violated under time pressure.

29. Data Flow

Primary flow — a shipment document, start to finish:

1. Source: User's browser (file upload)
2. Transformation 1: Raw file → stored in object storage (encrypted at rest)
3. Transformation 2: Raw file → OCR text (if scanned) or direct text (if native PDF)
4. Transformation 3: Raw text → structured fields (LLM extraction)
5. Transformation 4: Structured fields → HS classification, consistency checks, permit triage, CEPA determination
6. Destination: All results persisted to PostgreSQL, linked to the shipment record
7. Final output: Compiled into a PDF report, stored in object storage, made available for download

Deletion: On account closure/deletion request, shipment documents and derived data are deleted from both the database and object storage within a defined window (ASSUMPTION: 30 days, pending confirmation of any regulatory retention requirement that might override user-requested deletion — e.g., the 5-year VAT record-keeping consideration noted in Section 9 needs legal confirmation before finalizing this policy).

30. State Management

Shipment processing state machine:

```
stateDiagram-v2  
    [*] --> Created  
    Created --> DocumentsUploading
```

```
DocumentsUploading --> Extracting
Extracting --> NeedsManualReview: low confidence
Extracting --> Classifying: extraction succeeds
Classifying --> NeedsManualReview: low confidence
Classifying --> AnalysisComplete: confidence sufficient
NeedsManualReview --> AnalysisComplete: user resolves
AnalysisComplete --> ReportGenerated
ReportGenerated --> [*]
```

Persisted as a `status` field on the `Shipment` record; transitions are enforced in the service layer (never allow a skip from `Created` directly to `AnalysisComplete` , for example) to keep the state machine’s guarantees meaningful.

31. Edge Cases and Failure Modes

Failure mode	Detection	Handling	User impact
LLM returns malformed/unparseable output	Schema validation on LLM response	Retry once; if still malformed, mark <code>needs_manual_review</code>	Item flagged, not silently wrong
Duplicate document upload	Idempotency key check	Return existing result, no reprocessing	None — transparent
Concurrent edits to the same shipment (broker + client both editing)	Optimistic locking (version column)	Last-write conflict surfaced to user, not silently overwritten	User asked to refresh/reconcile
Worker crash mid-task	Celery’s built-in task acknowledgment	Task requeued automatically	Slight delay, no data loss
Corrupt/unreadable uploaded file	File validation on upload	Reject immediately with clear error, before queueing	Immediate, clear feedback
HS code genuinely ambiguous (multiple equally valid classifications)	Low confidence gap between top candidates	Present both, explicitly labeled “ambiguous — professional review recommended”	Transparent about tool’s limits — critical for a compliance-adjacent product

Regulatory rules table goes stale	No automatic detection (data-driven risk)	Manual quarterly review process (operational, not engineering)	Risk explicitly flagged, not fully engineerable away
-----------------------------------	---	--	--

32. Future Scalability and Extensibility

Reasonable near-term extensions once traction justifies them:

- Splitting the Classification module into an independently-scaled service if LLM-call volume outgrows the rest of the system's needs
- Direct Mirsal 2 filing integration (post government-relationship-building, per ADR-002)
- Additional CEPA agreements beyond the initial curated set
- API access for enterprise customers wanting ERP integration
- Multi-country expansion beyond UAE (architecture's tenant model and rules-engine pattern generalize reasonably well to this)

Not designed for yet, and explicitly deferred: multi-region active-active deployment, event-driven architecture, multi-tenancy beyond the company/broker model already described.

33. Implementation Plan

Given the "full scope" MVP decision combined with a solo founder, the phases below sequence the work to get to a demoable, sellable product as early as possible, even though the ultimate goal covers all four capabilities.

```

Phase 1 → Foundation (auth, company/user model, document upload + storage)
Phase 2 → Document extraction pipeline (OCR + LLM field extraction)
Phase 3 → HS classification (the core differentiator – build and validate this fi
Phase 4 → Consistency checking (relatively fast to build once extraction pipeline
Phase 5 → Permit triage (requires building the rules database – start this data-c
Phase 6 → CEPA origin qualification (most complex rules logic – sequence last amo
Phase 7 → Report generation + billing + polish
Phase 8 → Pilot customer onboarding + feedback loop

```

Phase 3 detail (as an example of the expected depth):

- Objective: Working HS classification with confidence scoring

- **Components:** `ClassificationService`, `LLMClient`, `TariffKBRepository`, `tariff`
schedule data ingestion (one-time ETL to populate the vector store from the published UAE/GCC tariff schedule)
- **Dependencies:** Phase 2 must be complete (need extracted product descriptions as input)
- **Testing requirement:** Golden-set regression suite must exist and pass before this phase is considered done
- **Definition of done:** 10 real (anonymized/sample) invoices processed end-to-end with classification accuracy validated against manually-verified correct codes

Sequencing note: Permit-rules data collection (Phase 5) is largely research/data-entry work, not blocked by engineering — it's reasonable to start this in parallel with Phase 3/4 development, since it's the same kind of research already done for the document checklist produced earlier in this project.

34. File-by-File Implementation Blueprint (key files)

`backend/app/services/classification_service.py`

Purpose: Orchestrates HS code classification for a line item.

Responsibilities:

- Accept a `ProductLineItem`
- Query `TariffKBRepository` for candidate headings via vector search
- Call `LLMClient` with candidates + GRI reasoning prompt
- Parse and validate LLM output against expected schema
- Apply confidence threshold logic
- Return a `ClassificationResult`

Dependencies: `TariffKBRepository`, `LLMClient`, `config` (confidence threshold setting)

`backend/app/services/permit_triage_service.py`

Purpose: Determines applicable UAE regulators for a shipment's HS codes.

Responsibilities:

- Accept a list of HS codes
- Query `PermitRulesRepository` (backed by `data/permit_rules/*.yaml`)
- Return a list of `PermitRequirement` objects, deduplicated across line items

Dependencies: `PermitRulesRepository`

Notes: This file's logic is simple; its correctness depends entirely on the maint

backend/app/workers/document_processing_tasks.py

Purpose: Celery task chain for document upload → extraction → classification → tr

Responsibilities:

- `process_document(document_id)`: OCR + extraction
- `classify_shipment(shipment_id)`: triggered once all documents extracted
- `run_consistency_check(shipment_id)`
- `run_permit_and_cepa(shipment_id)`

Dependencies: All relevant services; DocumentRepository, ShipmentRepository

Notes: Tasks are chained via Celery's ``chain()`` primitive so failures at one stag

35. Coding Standards and Architectural Rules

- Naming: `snake_case` for Python, `camelCase` for TypeScript/frontend, consistent across the codebase
 - Module boundaries and dependency direction: strictly as defined in Sections 5, 7, 28 — enforced via CI lint rule, not just convention
 - Error handling: never swallow an exception silently; every `except` block either re-raises, logs with context, or explicitly handles and returns a defined failure state
 - Logging: structured, never log secrets or raw sensitive document content (Section 17)
 - API conventions: consistent error response shape (Section 16), versioned routes (`/api/v1/...`) from day one to allow future breaking changes without disrupting existing integrations
 - Type safety: full type hints in Python (mypy-checked in CI), strict TypeScript on the frontend
 - Testing: no PR merges without passing the golden-set regression suite for anything touching classification logic
 - Documentation: every service's public methods have docstrings explaining purpose, inputs, outputs, and failure modes — mirroring the level of detail in Section 6 of this document
-

36. "Do Not Do This" Section

- Do not put business logic in API route handlers — they validate and delegate only

- Do not call the database directly from a service — always go through a repository
 - Do not let the classification module silently return a low-confidence guess as if it were certain — always surface uncertainty
 - Do not hardcode permit/CEPA rules as `if` statements scattered through code — keep them in the versioned data files (Section 23) so they're auditable and updatable without a code deploy
 - Do not introduce microservices, Kubernetes, or a separate vector database before there's a measured, specific performance reason to (Sections 4, 9, 26)
 - Do not auto-file anything with a government system without explicit, logged user confirmation — this product gives advice, it does not act on the user's behalf at MVP (ADR-002)
 - Do not log raw document content or full LLM prompts containing sensitive commercial data
 - Do not treat a stale permit-rules table as a code problem to "fix later" — it's an ongoing compliance-data maintenance responsibility that needs a recurring review process, not a one-time engineering task
-

37. Open Questions and Assumptions

Confirmed requirements

- Full-scope MVP (classification + consistency + permit triage + CEPA) per founder's explicit choice
- Solo founder, no team yet
- Founder chooses tech stack (delegated to this document's recommendations)

Assumptions (all flagged inline above, consolidated here)

- No legally-binding classification liability at MVP — advisory framing only
- UAE/GCC-region hosting preferred but not confirmed as a hard legal requirement
- 5-year document retention target, pending legal confirmation
- Permit rules maintained manually/periodically, not live-scraped
- CEPA coverage starts with a curated subset of agreements, not all 20+

Open questions (need founder input or legal consultation before finalizing)

- Exact data residency/retention legal requirements for UAE trade-compliance data
- Whether any liability/insurance structure is needed before offering classification “advice” commercially
- Which specific CEPA agreements to prioritize first (recommend validating against actual pilot customers’ trade lanes)
- Pricing model details (per-shipment vs. flat subscription vs. tiered) — not an architecture concern directly, but affects the Billing module’s data model

Risks

- Solo founder attempting full scope simultaneously is ambitious — Section 33’s phasing exists specifically to de-risk this by sequencing rather than attempting all four modules in parallel
- Permit-rules and CEPA-rules accuracy is an ongoing compliance-data maintenance burden, not a one-time build — underestimating this is the single biggest non-technical risk to the product’s credibility
- LLM cost per classification could be a meaningful unit-economic risk at scale — worth instrumenting (Section 17) from day one

38. Final Recommended Architecture

Stack: Python/FastAPI backend (modular monolith) + Next.js frontend + PostgreSQL/pgvector + Redis/Celery for async processing + S3-compatible object storage + an LLM provider API for extraction/classification reasoning.

Main components: API layer, five domain services (Classification, Consistency, Permit Triage, CEPA, Reports) plus Billing, all behind a thin controller layer, backed by per-aggregate repositories.

Data stores: One PostgreSQL instance (relational + vector search via pgvector), one object store for documents/reports.

Communication: Synchronous REST for user-facing requests; async task queue for anything touching an external AI/OCR API.

Security model: JWT auth, strict tenant isolation enforced at the repository layer, encryption in transit/at rest, prompt-injection-aware LLM integration.

Deployment model: Single-region containerized deployment on a managed platform, no Kubernetes, staged rollout (staging → manual-gated production).

Repository structure: Modular monolith with strict one-directional dependency flow: `api` → `services` → `repositories` → `models`, mirrored by a `workers` entry point for async tasks.

Implementation phases: Foundation → extraction pipeline → classification (core differentiator, built and validated first) → consistency checking → permit triage → CEPA → reporting/billing → pilot onboarding.

This architecture is deliberately sized for one technical founder to build and operate, while preserving clear seams (module boundaries, the `LLMClient` interface, the data-driven rules engines) that make it possible to scale up complexity — more team members, split-out services, additional jurisdictions — only once real usage data justifies the added operational cost.

39. Implementation-Oriented, Not Generic — Self-Check

This document is checked against the drill-down chain it's meant to satisfy:

```
System (Sijil platform)
↓
Subsystem (Classification / Consistency / Permit Triage / CEPA / Reports / Billin
↓
Service (ClassificationService, PermitTriageService, ...)
↓
Module (classification_service.py, permit_rules_repository.py, ...)
↓
Class / Interface (LLMClient protocol, repository classes)
↓
Function (classify(), vector_search(), ...)
↓
Data / API Contract (ClassificationResult schema, POST /shipments/{id}/documents)
```

Every subsystem in Section 4 is traceable down to named files in Section 7, named methods in Sections 6 and 8, and concrete request/response shapes in Section 10 — a developer can go from “what does the system do” to “which file do I open” without an intermediate translation step.

40. Explaining “Why” — Where the Reasoning Lives

Every non-trivial decision in this document is paired with its reasoning, not just its conclusion, so the document holds up months later:

- **Section 4** — modular monolith over microservices, justified by solo-founder operating capacity, not asserted as a default
- **Section 9** — pgvector inside Postgres instead of a dedicated vector database, justified by avoiding a second operational system at this data volume
- **Section 19** — every technology choice lists rejected alternatives and why they lost
- **Section 20 (ADRs)** — the three highest-stakes decisions (monolith vs. microservices, advisory tool vs. direct Mirsal 2 filing, rules-engine vs. live scraping) each carry full context/options/trade-offs/consequences
- **Section 36** — every “don’t do this” rule is a decision with a reason, not a bare prohibition

Nowhere in this document is a technology or pattern chosen because it’s fashionable — each choice traces back to a stated constraint (solo founder, MVP scope, cost sensitivity, or a specific UAE regulatory reality from the original research).

41. Use of Mermaid — Index

Diagrams are placed at the point where prose alone would leave the structure ambiguous, not decoratively:

Diagram	Section	Purpose
System architecture flowchart	4	Component-level map of the whole platform
Class diagram	8	<code>ClassificationService</code> and its dependencies
Entity-relationship diagram	9	Full data model
Sequence diagram	11.1	Document processing workflow, success and failure paths
Deployment flowchart	24	Runtime topology from user to managed services
State diagram	30	Shipment processing lifecycle

No diagram was added purely for visual filler — each one is referenced by the surrounding prose and would leave a real gap in understanding if removed.

42. Avoiding Overengineering — Explicit Record

Given a solo founder building this, the following were deliberately **not** introduced, with the specific reasoning recorded rather than left implicit:

- **No microservices** (Section 4, ADR-001) — one person can't operate a service mesh while also building product
- **No Kubernetes** (Sections 24, 26) — a single managed container platform is sufficient at ~500 companies / 5,000 shipments a month
- **No dedicated vector database** (Section 9, Section 19) — pgvector inside the existing Postgres instance covers the required scale
- **No event-driven architecture** (Section 32) — Celery task chains are sufficient for the async needs identified; a message bus would add operational surface with no corresponding requirement
- **No multi-region/active-active deployment** (Sections 26, 32) — not justified until traffic and revenue say otherwise
- **No speculative interfaces** — only `LLMClient` is abstracted (Section 8), because it's the one component with a real, near-term reason to be swapped; repositories and other services are implemented concretely

Each of these is a "not yet," not a "never" — Section 32 records the specific trigger conditions (traction, team growth, measured load) under which they'd be revisited.

43. Output Format — Compliance Note

This document is delivered as a single Markdown file

(`SOFTWARE_ARCHITECTURE_AND_TECHNICAL_DESIGN.md`), using consistent heading hierarchy, tables for structured comparisons (Sections 3, 19, 27, 31), Mermaid diagrams where noted in Section 41, code/directory-tree blocks for concrete structure (Sections 7, 8, 34), and explicit JSON examples for API contracts (Section 10). No section was silently omitted — where a topic didn't apply at full strength (e.g., service-to-service auth, Section 14), that was stated directly rather than left blank.

44. Critical Thinking Requirement — Findings

Reasoning through this architecture before finalizing surfaced the following, each already reflected in the relevant section rather than left as a hidden risk:

- **Contradictory pressure:** “full scope” MVP vs. “solo founder” is inherently in tension — resolved not by cutting scope (the founder’s explicit choice) but by sequencing (Section 33) so no phase requires parallel expertise the founder doesn’t have alone
- **Missing requirement surfaced during design:** the permit-rules and CEPA-rules tables need an ongoing *maintenance process*, not just an initial data load — this isn’t a coding task and is called out separately in Sections 6.3, 20 (ADR-003), 31, 36, and 37 so it doesn’t get silently treated as “done” once the initial rules are entered
- **Security issue surfaced:** passing extracted document text into LLM prompts creates a prompt-injection surface — addressed directly in Section 15 rather than assumed away
- **Consistency risk:** a naive “does not qualify” default on missing CEPA origin data would cost users real money on false negatives — Section 2.5 and 6.4 explicitly require requesting missing data instead of defaulting
- **Architectural coupling risk:** solo-founder time pressure is exactly the condition under which module boundaries erode — Section 28 recommends a CI-enforced import-linter rule rather than relying on discipline alone
- **Technology mismatch avoided:** an initial instinct toward a dedicated vector DB and microservices was checked against actual projected scale (Section 3) and rejected as premature (Sections 9, 42)

Where uncertainty remained genuinely unresolved (legal retention requirements, liability structure, exact pricing model), it’s listed as an open question in Section 37 rather than papered over with an invented answer.

45. Source of Truth — Position in the Documentation Hierarchy

```
PRD
|   What are we building? (captured in Section 1 pending a standalone PRD.md)
▼
REQUIREMENTS
|   Sections 2-3
▼
SYSTEM DESIGN
|   Sections 4-5
```

```

▼
ARCHITECTURE
| Sections 6-20
▼
COMPONENT DESIGN
| Sections 6, 8, 34
▼
CODE STRUCTURE
| Section 7
▼
IMPLEMENTATION PLAN
| Section 33
▼
CODE

```

This document is the bridge in the middle of that chain — it assumes the requirements captured informally in Section 1 (no separate `PRD.md` exists yet) and hands off to actual implementation via Sections 7, 33, and 34. If a standalone PRD is written later, Section 1 of this document should be trimmed to reference it rather than duplicating it.

46. Relationship to Other Project Documents

```

docs/
├── PRD.md                                ← not yet written; Section 1
├── SOFTWARE_ARCHITECTURE_AND_TECHNICAL_DESIGN.md ← this document
├── API_SPEC.md                          ← not yet written; Section 10
└── IMPLEMENTATION_PLAN.md               ← not yet written; Section 33

```

Document	Purpose	Current status
PRD.md	What are we building and why	Not yet a standalone file — covered in Section 1
SOFTWARE_ARCHITECTURE_AND_TECHNICAL_DESIGN.md	How the system works and maps to code	This document
		Not yet a standalone

API_SPEC.md	Full endpoint-by-endpoint contract	file — Section 10 covers the core routes; a dedicated spec (e.g., OpenAPI schema) is worth generating once the API stabilizes past MVP
IMPLEMENTATION_PLAN.md	Step-by-step build plan	Not yet a standalone file — Sections 33–34 cover this today

As the project matures past MVP, splitting Sections 1, 10, and 33–34 out into their own dedicated files is reasonable — this document doesn't need to stay the single source for everything forever, just for now.

47. Final Quality Bar — Verification

Checked against the 30-question bar this document is meant to satisfy:

- **What/why/functional/non-functional requirements** — Sections 1–3
- **Architecture, components, communication** — Sections 4–5
- **Data model, APIs, workflows, data flow** — Sections 9–11, 29
- **Auth, authorization, security** — Sections 14–15
- **Error handling, testing, monitoring** — Sections 16–17, 21
- **Scaling and deployment** — Sections 18, 24, 26
- **Repository structure, modules, files, dependencies** — Sections 7–8, 28, 34
- **Decisions and reasoning** — Sections 19–20, 40
- **Edge cases and failure modes** — Section 31
- **Step-by-step implementation order** — Section 33
- **Assumptions and open questions surfaced, not hidden** — Section 37

The one honest caveat: this document can only be a true blueprint up to the limits of what's currently confirmed. Sections 1 and 37 are explicit about where founder input or legal review is still needed (data residency, retention, liability structure, CEPA agreement prioritization)

— those gaps are named rather than filled with invented certainty, per Section 44's findings. Everything else is implementation-ready as written.